🌱 COMPREHENSIVE TECHNICAL ARCHITECTURE & PROJECT GUIDE

SmartKisan

An AI-Powered Agricultural Advisory Platform for Indian Farmers — Combining Deep Learning Computer Vision, Agronomic Recommendation Engines, Real-Time Mandi Intelligence, and Multilingual Generative AI.

- 🍃 EfficientNetB3 Disease Detection
- 🌱 Random Forest Crop Recommendation
- 🌼 Open-Meteo Weather Advisory
- 📊 APMC Mandi Price Intelligence
- 🏛️ National Schemes Directory
- 💬 Gemini 1.5 Flash Multilingual RAG

BACKEND ARCHITECTURE
FastAPI + Python 3.13

FRONTEND STACK
React 19 + Vite

DATABASE LAYER
SQLAlchemy + SQLite / PostgreSQL

LANGUAGES SUPPORTED
English, Hindi, Marathi

1 Project Overview & Core Mission

Smart Kisan ("Smart Farmer") is an end-to-end, full-stack, artificial intelligence-powered agricultural advisory platform created specifically to address the structural technology divide in Indian agriculture. Over 140 million farming households in India face recurring agricultural and economic vulnerabilities due to five fundamental challenges:

1. ASYMMETRIC CROP HEALTH INFORMATION

Farmers often fail to diagnose fungal and bacterial leaf diseases until irreversible damage occurs. Agricultural extension officers (Krishi Vigyan Kendras) are understaffed and geographically distant from remote villages.

2. SUBOPTIMAL CROP SELECTION

Sowing choices are frequently based on tradition or speculative market hype rather than objective soil chemistry (Nitrogen, Phosphorus, Potassium, pH) and seasonal meteorological forecasts.

3. MARKET MANDI PRICE EXPLOITATION

Middlemen (arhtiyas) leverage price opacity in regional Agricultural Produce Market Committees (APMCs) to purchase farmer harvests well below prevailing modal market rates.

4. UNDERUTILIZATION OF GOVERNMENT WELFARE

Billions of rupees allocated under PM-KISAN, PMFBY (Crop Insurance), and Kisan Credit Card (KCC) remain unclaimed because eligibility rules are buried in complex government portals.

THE SMART KISAN SOLUTION

Smart Kisan consolidates six distinct agricultural domains into a single, high-performance, mobile-first web ecosystem accessible across desktop, tablet, and smartphone browsers in the farmer's native tongue (English, Hindi, and Marathi):

- **Instant Leaf Disease Diagnosis:** Point-and-shoot computer vision diagnostics delivering actionable chemical treatments, preventative measures, and KVK contact recommendations within seconds.
- **Agronomic Crop Advisor:** Scientific crop recommendations matching exact soil nutrient parameters (NPK, pH) and micro-climate inputs with regional soil presets.
- **Real-time Mandi Intelligence:** Transparent pricing across major Indian markets with automatic "Best Mandi" profit optimization highlights.
- **Precision Meteorological Alerts:** 7-day hyperlocal forecast paired with rule-based operational advisories (foliar spray timing, rain shelter alerts).
- **National Welfare Directory:** Searchable repository of subsidies, loans, and insurance schemes with direct application pathways.
- **Conversational Krishi Mitra AI:** 24/7 multilingual agronomic intelligence grounded in ICAR research through Retrieval-Augmented Generation (RAG).

7	38+	22
PRODUCTION MODULES	CROP DISEASE CLASSES	ANALYZED CROP VARIETIES

2 System Architecture & Data Flow

Smart Kisan adheres to a modern, decoupled 3-tier web architecture engineered for speed, resilience, and offline accessibility:

System Architecture Diagram



Request Lifecycle & Execution Flow

1. **Client Dispatch:** The user interacts with the UI. The frontend service module (`frontend/src/services/api.js`) intercepts the request and automatically attaches the JWT bearer token from `localStorage` if present.
2. **Reverse Proxy / CORS:** In development, the Vite server proxies API calls from port 5173 to port 8000. In production, FastAPI's CORS middleware ensures only authorized domains can access backend endpoints.

3. **Security & Rate Limiting:** SlowAPI inspects the client IP address. High-compute endpoints (such as image disease prediction) are capped at 10 requests per minute to prevent resource exhaustion.
4. **Authentication Resolution:** `get_current_user` verifies token signatures using HS256. If no token is provided, requests still succeed gracefully via the built-in *Guest Mode* fallback.
5. **Business Logic & Inference:** The router triggers the corresponding service. ML models are loaded lazily into memory on the first request to eliminate cold-start server lag.
6. **Database Transaction:** Queries, caches, and records are committed via SQLAlchemy sessions and returned through Pydantic response models.

3 Technology Stack & Architectural Rationale

Every tool and library in Smart Kisan was selected specifically for high throughput, minimal developer friction, low hosting cost, and resilience under erratic rural internet connectivity.

1. Backend Engineering (Python Ecosystem)

PACKAGE / TOOL	VERSION	ROLE & ARCHITECTURAL RATIONALE
`fastapi`	0.115.0	High-performance asynchronous web framework based on Starlette and Pydantic. Auto-generates OpenAPI / Swagger schemas, enforces strict type validation, and supports async I/O.
`uvicorn[standard]`	0.30.6	Lightning-fast ASGI web server implementation using uvloop and httptools. Powers development hot-reloading (`--reload`).
`sqlalchemy`	2.0.35	Production-grade Object Relational Mapper (ORM). Decouples database models from the underlying SQL dialect, enabling seamless migration from SQLite to PostgreSQL.
`pydantic`	2.9.2	Data modeling and parsing library. Ensures incoming HTTP JSON payloads conform strictly to domain expectations before reaching router logic.
`python-jose`	3.3.0	Cryptographic library for JSON Web Tokens (JWT). Signs and verifies user identity tokens using SHA-256 HMAC (HS256) with 24-hour expiration windows.
`passlib[bcrypt]`	1.7.4	Industry standard password hashing with automatic salt generation and timing-attack resistant verification.
`tensorflow`	2.17.0	Executes the pre-trained EfficientNetB3 deep learning convolutional neural network for plant disease inference.
`scikit-learn`	1.5.2	Executes the Random Forest classifier for crop recommendation and handles feature normalization with `StandardScaler` .
`google-generativeai`	0.8.2	Official SDK for Google Gemini 1.5 Flash. Delivers sub-second generative responses with high multilingual translation fidelity.
`chromadb`	0.5.7	High-performance vector database. Computes cosine similarity across indexed agronomic documents to provide grounded context for RAG.
`slowapi`	0.1.9	IP-based rate limiting middleware. Guards against denial-of-service attempts and malicious bot scraping on heavy ML endpoints.
`httpx`	0.27.2	Asynchronous HTTP client for non-blocking outbound requests to Open-Meteo and data.gov.in APIs.

2. Frontend Engineering (JavaScript Ecosystem)

PACKAGE / TOOL	VERSION	ROLE & ARCHITECTURAL RATIONALE
`react`	19.2.8	Declarative component-based UI framework leveraging concurrent rendering and modern hook patterns (`useState` , `useEffect` , `useContext`).

`vite`	8.3.0	Next-generation frontend tooling providing sub-100ms Hot Module Replacement (HMR) and optimized Rollup asset bundling.
`react-router-dom`	7.18.4	Client-side routing managing URL transitions across all 7 feature modules without full browser reloads.
`index.css`	Custom	Comprehensive 58KB Vanilla CSS Design System. Avoids bulky runtime CSS-in-JS dependencies while enforcing harmonious green-amber design tokens.
`i18next`	26.4.2	Complete internationalization engine holding 200+ localized agricultural phrases in English, Hindi, and Marathi with zero remote fetch latency.
`recharts`	3.10.1	SVG charting library. Renders responsive bar graphs comparing commodity prices across Indian APMC mandis.
`lucide-react`	1.52.0	Crisp SVG iconography enhancing visual navigation for semi-literate rural users.

4 Project Directory & File Structure

The repository follows a clean separation between backend service APIs, frontend client pages, machine learning assets, and database seeds:

Project Repository Layout

```
Smart-Kisan/
├── .env.example                    # Environment template defining DB, SECRET_KEY, API keys
├── README.md                       # Developer onboarding and system launch manual
├── smart_kisan.db                  # Root SQLite database holding persistent records
│
├── backend/                        # FastAPI Application Package
│   ├── requirements.txt            # Manifest of 22 Python dependencies
│   └── app/
│       ├── main.py                 # Server entry point: middleware, routers, lifespans
│       ├── config.py               # Pydantic Settings reading .env configurations
│       ├── database.py             # Engine creation, declarative base, get_db() generator
│       ├── models/
│       │   └── __init__.py         # Declarative SQLAlchemy models (Users, Diseases, Crops, etc.)
│       ├── schemas/
│       │   └── auth.py             # Pydantic schemas validating registration & login
│       ├── core/
│       │   └── security.py         # Bcrypt hashing utilities, JWT encoder/decoder, auth dependency
│       ├── routers/                # Modular REST route controllers
│       │   ├── auth.py             # /api/v1/auth (register, login, me, profile updates)
│       │   ├── disease.py          # /api/v1/predict/disease (image upload & diagnosis)
│       │   ├── crop.py             # /api/v1/recommend/crop (NPK agronomic recommendations)
│       │   ├── weather.py          # /api/v1/weather (Open-Meteo fetch & advisory engine)
│       │   ├── market.py           # /api/v1/market (APMC mandi pricing & historical trends)
│       │   ├── schemes.py          # /api/v1/schemes (search & filter central/state schemes)
│       │   ├── chat.py             # /api/v1/chat (multilingual Gemini RAG interaction)
│       │   └── history.py          # /api/v1/history (unified farmer diagnostic history)
│       ├── ml/                     # Machine Learning inference engines
│       │   ├── disease_model.py    # EfficientNetB3 loader, image preprocessor, predictor
│       │   ├── crop_model.py       # Random Forest joblib loader, scaler, prediction logic
│       │   └── rag_engine.py       # ChromaDB vector indexer & Gemini client integration
│       └── utils/
│           └── weather_rules.py    # Agronomic heuristic engine generating weather advisories
│
├── frontend/                       # React 19 Client Application
│   ├── package.json                # Node dependencies and build scripts
│   ├── vite.config.js              # Dev server configuration with backend /api proxy
│   └── src/
│       ├── App.jsx                 # Root layout, routing routes, Splash and Auth wrapper
│       ├── i18n.js                 # 200+ translation keys for English, Hindi, Marathi
│       ├── index.css               # 58KB Vanilla CSS design system with custom properties
│       ├── context/
│       │   └── AuthContext.jsx     # Global authentication state, JWT storage, Guest Mode
│       ├── services/
│       │   └── api.js              # Axios client with automatic Bearer token interceptor
│       ├── components/
│       │   ├── Sidebar.jsx         # Desktop responsive sidebar navigation
│       │   ├── TopHeader.jsx       # Header with breadcrumb trail, mobile toggle, language
│       │   ├── BottomNav.jsx       # Mobile 5-tab fixed navigation dock
│       │   └── SplashScreen.jsx    # 1.9-second animated branding introduction
│       └── pages/                  # Core view components
│           ├── Home.jsx            # Dashboard summary, quick actions, weather overview
```

```
│           ├── DiseaseDetector.jsx    # Leaf image uploader, camera capture, diagnosis report
│           ├── CropAdvisor.jsx        # Soil NPK form, regional presets, crop ranking view
│           ├── MarketPrices.jsx       # Commodity selection, APMC table, Recharts chart
│           ├── Schemes.jsx            # Government scheme cards, category filters, search
│           ├── Chat.jsx               # Conversational AI dialogue interface with audio/chips
│           └── Profile.jsx            # Farm data, language settings, consolidated logs
│
├── database/
│   └── seed_schemes.py              # Seeds 10+ official farmer welfare schemes
├── models/                          # Binary weights (EfficientNet, Random Forest, ChromaDB)
├── uploads/                         # Persistent storage for uploaded leaf diagnosis photos
└── docs/                            # Project manuals, architectural diagrams, PDF documentation
```

5 In-Depth Feature Breakdown

Smart Kisan is built around 7 core modules, each engineered with strict validation, real-time feedback, and offline fallback capability:

1. Leaf Disease Doctor (`/disease`)

Allows farmers to upload or snap a photo of an infected leaf. The platform executes an AI visual diagnostic and returns a tailored disease report.

- **Input:** Image file (JPEG, PNG, WebP) up to 10MB via file picker or webcam.
- **File Validation:** Validates MIME headers and inspects raw magic bytes (`\xFF\xD8\xFF` for JPEG, `\x89PNG` for PNG, `RIFF` for WebP) to prevent malicious executable injection.
- **Inference Engine:** Resizes image to 224×224 RGB, applies ImageNet normalization, and runs EfficientNetB3 inference across 38+ plant pathology classes.
- **Output:** Top-3 predicted diseases with confidence scores, severity rating (None, Medium, High), chemical treatments, cultural preventative actions, and a Krishi Vigyan Kendra (KVK) advisory notice.
- **Built-in Samples:** Includes instant 1-click test leaves (Tomato Early Blight, Healthy Potato, Corn Rust) for seamless demonstrations.

2. Crop Advisor (`/crop`)

Predicts the top 5 most profitable and biologically suitable crops based on soil chemistry and regional weather parameters.

- **Inputs:** Nitrogen (0–200 kg/ha), Phosphorus (0–200 kg/ha), Potassium (0–200 kg/ha), pH (0–14), Temperature (°C), Humidity (%), and Annual Rainfall (mm).
- **Regional Soil Presets:** Quick-fill buttons for *Black Cotton Soil* (High K, medium N/P), *Alluvial Soil* (Balanced NPK), and *Red Laterite* (Acidic, low N/P).
- **Inference Engine:** Standardizes input features via `StandardScaler` and evaluates them using a Random Forest Classifier trained on ICAR multi-crop benchmarks.
- **Output:** Ranked list of 5 crops with suitability percentages, water requirements (Low/Medium/High), optimal growing season (Kharif, Rabi, Zaid), and current market demand.

3. Weather Advisory & Alert Engine (`/`)

Integrates precision meteorological data with agronomic heuristic rules to safeguard crops against adverse weather events.

- **Data Provider:** Open-Meteo REST API. Completely free, high-uptime, requiring zero API keys.
- **Parameters:** 7-day temperature range (min/max), relative humidity, precipitation probability, wind speed, and reference evapotranspiration (ET_0).
- **Agronomic Rule Engine (`weather_rules.py`):**

- *Foliar Spray Advisory:* Flags high winds (>15 km/h) or impending rain (>40%) to warn against wasted pesticide application.
- *Crop Inundation Warning:* Alerts farmers when rainfall exceeds 50mm within 24 hours.
- *Frost Protection Alert:* Warns northern farmers when night temperatures dip below 4°C.

- **Intelligent Cache:** Caches weather responses in the database for 6 hours per coordinate pair to eliminate redundant network traffic.

4. Live Mandi Market Prices (`/market`)

Empowers farmers to negotiate fair prices by providing real-time commodity pricing across Agricultural Produce Market Committees (APMCs).

- **Data Source:** data.gov.in official Open Government Data (OGD) platform with automated fallback to realistic price simulations.
- **Commodity Scope:** 23 primary Indian crops including Rice, Wheat, Cotton, Soybean, Onion, Tomato, Mustard, and Turmeric.
- **Market Intelligence:**
  - Displays Minimum, Maximum, and Modal (prevailing) prices per quintal (100 kg).
  - **"Best Mandi" Badge:** Highlights the local market offering the highest modal payout.
  - **Recharts Interactive Graph:** Renders visual price comparisons across state mandis.

5. Government Welfare Schemes Directory (`/schemes`)

Centralizes welfare programs, subsidies, and credit schemes from the Ministry of Agriculture and Farmers Welfare.

- **Cataloged Schemes:** PM-KISAN (£6,000 direct income support), PMFBY (Pradhan Mantri Fasal Bima Yojana - crop insurance), KCC (Kisan Credit Card at 4% subsidized interest), e-NAM (National Agriculture Market), and AIF (Agriculture Infrastructure Fund).
- **Filtering Dimensions:** Scheme type (Subsidy, Insurance, Credit, Organic), state applicability, and target farmer category (Marginal, Small, Medium).
- **Direct Action:** Full eligibility breakdowns, documentation requirements, and direct external links to official government application portals.

6. Krishi Mitra Multilingual AI Specialist (`/chat`)

A 24/7 agricultural domain chatbot powered by Google Gemini 1.5 Flash grounded with ChromaDB vector search.

- **Multilingual Engine:** Native conversational understanding and generation in **English**, **हिंदी (Hindi)**, and **मराठी (Marathi)**.
- **RAG Grounding:** Vector database contains verified agronomic advisories. Restricts the model from inventing unverified chemical dosages or false pricing.

- **Conversation Memory:** Retains dialogue history for multi-turn conversational follow-ups.

7. Farmer Profile & Consolidated History (`/profile`)

A centralized dashboard for personal farm metrics, past diagnostic reports, and application-wide preferences.

- **Farm Demographics:** Landholding acreage, primary cultivated crops, state/district, and soil classification.
- **Audit Logs:** Unified chronological timeline of previous leaf disease scans (with preview thumbnails) and crop recommendation runs.
- **Instant Localization:** Allows switching the active UI language across all pages with a single click.

6 Complete REST API Reference

All endpoints are prefixed with `/api/v1`. Interactive documentation is accessible at `http://127.0.0.1:8000/docs`.

Authentication Endpoints (`/auth`)

POST	`/auth/register`	Registers a farmer account with phone, full name, language, and password.
POST	`/auth/login`	Authenticates phone & password. Returns JWT access token (HS256).
GET	`/auth/me`	Fetches current profile. Requires `Authorization: Bearer <token>`.
POST	`/auth/profile`	Updates farm details (land size, crops, state, coordinates).

Plant Pathology Diagnostics (`/predict`)

POST	`/predict/disease`	Accepts multipart leaf image. Returns top-3 diagnoses, treatments, severity. (Rate: 10/min)
GET	`/predict/disease/history`	Returns paginated diagnostic history for the authenticated farmer.

Agronomic Crop Recommendations (`/recommend`)

POST	`/recommend/crop`	JSON payload: {N, P, K, pH, temperature, humidity, rainfall}. Returns top 5 crops.

Weather & Advisory (`/weather`)

GET	`/weather?lat=18.52&lon=73.85`	Fetches 7-day forecast & farming alerts. Cached 6 hrs per coordinate.

Mandi Market Rates (`/market`)

GET	`/market/commodities`	Returns list of all 23 trackable agricultural commodities.
GET	`/market/prices?commodity=Wheat&state=Punjab`	Fetches modal/min/max rates across APMCs with Best Mandi tag.

Welfare Schemes (`/schemes`)

GET	`/schemes?scheme_type=subsidy&q=kisan`	Searches and filters government welfare schemes.
GET	`/schemes/types`	Returns categorized scheme types with emoji icons and descriptions.

Conversational AI (`/chat`)

POST	`/chat`	Payload: {message, session_id, language}. Returns RAG-grounded response.

GET	`/chat/history`	Returns message history for active conversation session.

System & History (`/history` , `/health`)

GET	`/history?type=all`	Consolidated timeline of disease scans and crop recommendations.
GET	`/api/v1/health`	System health check: API version, database connectivity, model status.

7 Database Architecture & Data Models

Smart Kisan uses SQLAlchemy 2.0 ORM with 10 tables. The schema supports SQLite during local development and is fully compatible with PostgreSQL in production.

Primary Entities & Schema Definitions

TABLE NAME	PRIMARY & FOREIGN KEYS	CORE ATTRIBUTES & DATA TYPES
`users`	`id` (PK, UUID String) Index on `phone`	`phone` (VARCHAR 15, Unique), `password_hash` (VARCHAR 255), `full_name` (VARCHAR 100), `preferred_language` (VARCHAR 10), `is_active` (BOOLEAN), `created_at` (DATETIME), `last_login` (DATETIME)
`farmer_profiles`	`id` (PK, UUID) `user_id` (FK → users.id)	`state` (VARCHAR 50), `district` (VARCHAR 50), `latitude` (FLOAT), `longitude` (FLOAT), `land_size_acres` (FLOAT), `primary_crops` (JSON array), `farmer_category` (VARCHAR 20), `soil_type` (VARCHAR 50)
`disease_predictions`	`id` (PK, UUID) `user_id` (FK → users.id)	`image_path` (VARCHAR 500), `disease_name` (VARCHAR 150), `confidence` (FLOAT), `top3_results` (JSON array of predictions), `treatment_shown` (JSON array of action steps), `crop_type` (VARCHAR 100), `user_rating` (SMALLINT 1–5), `created_at` (DATETIME)
`crop_recommendations`	`id` (PK, UUID) `user_id` (FK → users.id)	`input_params` (JSON: N, P, K, pH, temp, humidity, rainfall), `recommended_crops` (JSON array of top 5 ranked crops), `created_at` (DATETIME)
`weather_cache`	`id` (PK, UUID) Index on `lat_lon_key`	`lat_lon_key` (VARCHAR 50: rounded lat/lon), `weather_data` (JSON: 7-day forecast), `farming_advice` (JSON: rules engine output), `expires_at` (DATETIME: 6h TTL)
`market_prices`	`id` (PK, UUID)	`commodity` (VARCHAR 50), `market_name` (VARCHAR 100), `state` (VARCHAR 50), `min_price` (FLOAT), `max_price` (FLOAT), `modal_price` (FLOAT), `price_date` (DATE), `source` (VARCHAR 20)
`government_schemes`	`id` (PK, UUID)	`scheme_name` (VARCHAR 150), `scheme_type` (VARCHAR 50), `description` (TEXT), `benefits` (JSON array), `eligibility_criteria` (JSON object), `applicable_states` (JSON array), `official_url` (VARCHAR 500)
`chat_history`	`id` (PK, UUID) `user_id` (FK → users.id)	`session_id` (VARCHAR 50), `role` (VARCHAR 20: user/assistant), `content` (TEXT), `language` (VARCHAR 10), `sources_used` (JSON array of citations), `created_at` (DATETIME)

`notifications`	`id` (PK, UUID) `user_id` (FK → users.id)	`type` (VARCHAR 50: weather/disease/scheme), `title` (VARCHAR 150), `message` (TEXT), `is_read` (BOOLEAN, default False), `created_at` (DATETIME)
`user_feedback`	`id` (PK, UUID) `user_id` (FK → users.id)	`feature` (VARCHAR 50), `prediction_id` (VARCHAR 50), `rating` (SMALLINT 1–5), `comment` (TEXT), `created_at` (DATETIME)

8 Machine Learning & AI Pipeline Deep Dive

The intelligence layer of Smart Kisan is split into three dedicated engines designed for fast inference and resilience under offline or uninstalled conditions:

1. Plant Pathology Vision: EfficientNetB3 (`disease_model.py`)

EfficientNetB3 scales depth, width, and resolution using a compound coefficient. It balances lightweight edge inference with high diagnostic accuracy on complex leaf textures.

Python Inference Pipeline

```
# 1. Lazy Loading (Loads into RAM only upon the first diagnostic request)
model = tf.keras.models.load_model("models/disease_model.keras")

# 2. Image Preprocessing
image = Image.open(BytesIO(image_bytes)).convert("RGB").resize((224, 224))
img_array = np.array(image, dtype=np.float32)
img_array = tf.keras.applications.efficientnet.preprocess_input(img_array)
img_array = np.expand_dims(img_array, axis=0)  # Shape: [1, 224, 224, 3]

# 3. Softmax Classification across 38+ plant pathology classes
raw_probs = model.predict(img_array, verbose=0)[0]
top3_idx = np.argsort(raw_probs)[-3:][::-1]
predictions = [{"disease": class_names[i], "confidence": float(raw_probs[i])} for i in top3_idx]
```

Fail-Safe Fallback: If TensorFlow binary weights are missing during local development, the system automatically falls back to high-fidelity mock diagnostic data (e.g. Tomato Late Blight at 87.3% confidence) to ensure the UI remains fully functional.

2. Agronomic Crop Recommendation: Random Forest (`crop_model.py`)

Trained on the ICAR Multi-Crop dataset across 2,200 soil-climate observations covering 22 distinct crops. Uses 100 decision trees to avoid overfitting on single soil parameters.

Crop Predictor Pipeline

```
# Feature vector order: [N, P, K, pH, temperature, humidity, rainfall]
raw_features = np.array([[n, p, k, ph, temp, humidity, rainfall]], dtype=np.float32)

# StandardScaler Normalization
scaled_features = scaler.transform(raw_features)

# Class Probability Estimation
probabilities = model.predict_proba(scaled_features)[0]
top5_indices = np.argsort(probabilities)[-5:][::-1]
recommendations = [
    {"crop": label_encoder.inverse_transform([idx])[0], "suitability": round(probabilities[idx] * 10
    for idx in top5_indices
```

```
]
```

3. Conversational RAG Engine: Gemini 1.5 Flash + ChromaDB (`rag_engine.py`)

Combines sub-second LLM generation with domain-verified agricultural publications stored in a local ChromaDB vector collection:

- **Vector Storage:** Indexed agricultural chunks from ICAR, extension bulletins, and government schemes using cosine similarity distance thresholds (`distance < 0.7`).
- **System Prompt Constraints:** Enforces domain isolation — declines medical, political, or off-topic questions; speaks in the farmer's dialect; explicitly recommends consulting local Krishi Vigyan Kendras for severe outbreaks.
- **Conversation Memory:** Contextually threads the last 3 conversational turns to maintain dialogue coherence during follow-up questions.

9 Frontend Architecture & Design System

The frontend is built with React 19 and Vite. It avoids third-party CSS frameworks like Tailwind or Bootstrap in favor of a bespoke 58KB Vanilla CSS design system (`index.css`) optimized for smooth rendering on low-cost Android devices.

HARMONIOUS DESIGN TOKENS

Built with agricultural tones: Deep Forest Green (`#14532d`), Leaf Emerald (`#22c55e`), Golden Harvest Amber (`#d97706`), and clean neutral grays for sunlight readability.

ADAPTIVE NAVIGATION

Desktop users get a permanent 7-item sidebar. Mobile devices switch to a fixed bottom 5-tab bar and a top bar with breadcrumb tracking.

Global Authentication & The "Guest Mode" Pattern

To prevent user friction in rural communities, Smart Kisan provides a **Guest Mode** managed by `AuthContext.jsx` :

- On load, the app checks `localStorage` for an existing JWT token.
- If present, it validates the token via `GET /api/v1/auth/me` and hydrates the farmer's profile.
- If absent, it automatically logs in a default guest user ("*Kisan Mitra (Demo)*"). Every feature remains accessible, ensuring zero onboarding friction for new visitors.

Internationalization (i18n.js)

All UI strings across all 7 views are stored directly in `frontend/src/i18n.js` . Farmers can toggle between English, Hindi, and Marathi in real time without refreshing the page or making network requests.

10 Security, Reliability & Quality Assurance

CRYPTOGRAPHIC TOKEN SECURITY

Authentication utilizes HS256 JWT tokens with a 24-hour expiration window. Passwords are salted and hashed using bcrypt via `passlib`.

MAGIC-BYTE UPLOAD VALIDATION

Prevents file extension spoofing by inspecting raw binary headers (JPEG, PNG, WebP) and enforcing a 10MB file size ceiling.

IP-BASED RATE LIMITING

SlowAPI enforces 10 requests per minute on deep learning inference routes, preventing denial-of-service from malicious actors.

SAFE ORM PARAMETERIZATION

SQLAlchemy 2.0 uses parameterized SQL queries, completely eliminating SQL injection risks across all endpoints.

11 Step-by-Step Developer Setup Guide

1. Prerequisites

Ensure Python 3.11+ (tested on Python 3.13) and Node.js 18+ with npm are installed on your workstation.

2. Clone Repository & Setup Environment

Bash / PowerShell

```
git clone https://github.com/your-username/Smart-Kisan.git
cd Smart-Kisan

# Copy environment variables template
copy .env.example .env    # Windows
# cp .env.example .env    # Linux / macOS
```

3. Launch Backend API

Bash / PowerShell (Backend Terminal)

```
cd backend
python -m venv venv
venv\Scripts\activate          # Windows (source venv/bin/activate on Linux/Mac)
pip install -r requirements.txt
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

The backend auto-seeds the database schema, seeds government welfare programs, and opens Swagger documentation at `http://127.0.0.1:8000/docs`.

4. Launch Frontend Web Application

Bash / PowerShell (Frontend Terminal)

```
cd frontend
npm install
npm run dev
```

Open `http://localhost:5173` in your browser to explore the running application.

12 Production Cloud Deployment Architecture

APPLICATION LAYER	RECOMMENDED HOST	CONFIGURATION DETAILS
FastAPI Backend	Render / AWS EC2 / Railway	Start command: `uvicorn app.main:app --host 0.0.0.0 --port $PORT`
React Frontend	Vercel / Cloudflare Pages	Build command: `npm run build`, Output directory: `dist`
Relational Database	Supabase / Neon (PostgreSQL)	Set `DATABASE_URL=postgresql://user:pass@host:5432/db` in environment variables